

Kapitel 9

Übersicht über die Standardbibliothek

Zu jeder C-Implementierung gehört standardmäßig eine Sammlung vordefinierter Routinen, die Ein-/Ausgabe und andere Operationen ermöglichen. Diese Sammlung wird *Standardbibliothek* genannt.

9.1 Headerdateien

Die Nutzung der Standardbibliothek erleichtern die *Standard-Headerdateien*, die der Standard vorschreibt, wenn eine Standardbibliothek vorhanden ist, und auf die man mit `#include`-Direktiven zugreifen kann. Man könnte die entsprechenden Deklarationen natürlich auch explizit in seine Programme schreiben – aber das wäre ziemlich unsinnig.

Insgesamt schreibt der Standard 15 Headerdateien vor. Einige davon haben wir bereits kennengelernt:

<code>ctype.h</code>	Zeichenverarbeitung
<code>float.h</code>	Interne Datenformate (Gleitkommatypen)
<code>limits.h</code>	Interne Datenformate (ganzzahlige Typen)
<code>stdio.h</code>	Ein-/Ausgabe
<code>string.h</code>	Stringverarbeitung

Einige weitere Headerdateien werden in diesem Abschnitt näher behandelt, auch wenn für eine vollständige Besprechung die Zeit fehlt:

<code>assert.h</code>	Testhilfen
<code>errno.h</code>	Fehlernummern
<code>math.h</code>	mathematische Funktionen
<code>stddef.h</code>	elementare Typen
<code>stdlib.h</code>	diverse Hilfsroutinen
<code>time.h</code>	Termine und Zeiten

Der Vollständigkeit halber seien auch die übrigen Headerdateien erwähnt:

<code>locale.h</code>	länderspezifische Darstellungen (Zeiten, Geldbeträge, usw.)
<code>setjmp.h</code>	Sprünge zwischen Funktionen
<code>signal.h</code>	Behandlung von Signalen („Interrupts“)
<code>stdarg.h</code>	Funktionen mit variabler Argumentzahl (wie <code>printf</code>)

Manche Deklarationen sind in mehreren Headerdateien enthalten. Das erlaubt es, alle Headerdateien unabhängig voneinander zu nutzen, und insbesondere, die `include`'s für

die Headerdateien in beliebiger Reihenfolge anzugeben. Andererseits erfordert es, die Headerdateien so gegeneinander abzusichern, dass der Compiler keine Deklarationen doppelt findet. Dieses ist mit bedingter Compilation möglich; die im nächsten Kapitel behandelt wird.

Für das Verständnis des weiteren ist es hilfreich daran zu erinnern, dass die Headerdateien die genannten Funktionalitäten nur bekannt machen. D.h. sie enthalten z.B. nur Prototypen von Funktionen. Dadurch wird es dem Compiler möglich, den korrekten Funktionsaufruf im Quelltext zu überprüfen. Die eigentlichen Implementationen können dagegen in Binärdateien gesammelt sein, die durch Vorkompilation oder anders erzeugt wurden.

9.2 Mathematische Funktionen

Die numerischen Funktionen, die man üblicherweise in Programmiersprachen zur Verfügung hat, bietet auch C. Die Prototypen sind in `math.h` enthalten.

Funktion	Beschreibung
<code>sin</code>	Sinus
<code>cos</code>	Cosinus
<code>tan</code>	Tangens
<code>asin</code>	Arcus Sinus
<code>acos</code>	Arcus Cosinus
<code>atan</code>	Arcus Tangens
<code>sinh</code>	Sinus hyperbolicus
<code>cosh</code>	Cosinus hyperbolicus
<code>tanh</code>	Tangens hyperbolicus
<code>exp</code>	Exponentialfunktion
<code>log</code>	natürlicher Logarithmus
<code>log10</code>	Logarithmus zur Basis 10
<code>sqrt</code>	Quadratwurzel
<code>ceil</code>	Aufrundung auf ganze Zahl (das Resultat ist ein <code>double</code> -Wert!)
<code>floor</code>	Abrundung auf ganze Zahl (das Resultat ist ein <code>double</code> -Wert!)
<code>fabs</code>	Absolutbetrag (Achtung: <code>abs</code> für <code>int</code> -Argumente und <code>labs</code> für <code>long</code> -Argumente sind in <code>stdlib.h</code> deklariert)

Tabelle 9.1: Mathematische Funktionen mit einem Argument

Viele der Funktionen haben die Deklaration

```
double name (double);
```

d.h. sie haben einen `double`-Parameter und liefern einen `double`-Funktionswert. Diese Funktionen sind in Tabelle 9.1 aufgelistet. Dazu ist noch wichtig zu erwähnen, dass die Winkelfunktionen alle im Bogenmaß (d.h. voller Kreisumfang entspricht 2π) rechnen. Von den übrigen Funktionen sind drei besonders erwähnenswert, die jeweils zwei `double`-Parameter besitzen und einen `double`-Funktionswert liefern:

`atan2` liefert bei einem Aufruf `atan2 (y, x)` den Arcus Tangens von y/x , wobei auch eines der beiden Argumente Null sein darf (aber nicht beide gleich-

zeitig). Der Wertebereich ist $[-\pi, \pi]$, unter Berücksichtigung der Vorzeichen beider Argumente, während der Wertebereich bei `atan` nur $[-\pi/2, \pi/2]$ ist.

fmod liefert bei einem Aufruf `fmod (x, y)` die Nachkommastellen des Quotienten $x/|y|$ ($y \neq 0$). Nützlich ist diese Funktion zum Beispiel, wenn man Winkel modulo 2π rechnen möchte.

pow liefert bei einem Aufruf `pow (x, y)` die Potenz x^y .

Der Standard spricht nur von *der* Standardbibliothek. Trotzdem können die Funktionen auf mehrere *Bibliotheksdateien* (d.h. Binärdateien die Implementationen von Funktionen enthalten) verteilt sein. Nicht immer durchsucht in solchen Fällen der Linker auch alle Bibliotheksdateien. Für diesen Fall und auch für die Möglichkeit der Erzeugung und des Testens eigener Bibliotheken gibt es Linker-Optionen, mit denen man die zu verwendenden Bibliotheksdateien angeben kann.

Speziell im Falle der Verwendung der mathematischen Bibliotheksfunktionen ist dies meist erforderlich und wird gern vergessen. Speziell im Falle des `gcc` benötigt man dazu die Option `-lm` im Aufruf des `gcc`, die *hinter* die Namen der Dateien geschrieben wird, etwa so:

```
gcc -Wall prog.c -lm
gcc -Wall -o prog prog.c -lm
```

Hintergrund: Bei einigen Rechnern beherrscht der Hauptprozessor selbst nur ganzzahlige Arithmetik. Für Gleitkommaarithmetik gibt es zwei Möglichkeiten: Man kann einen zusätzlichen Gleitkommaprozessor einbauen oder man muss die Gleitkommaarithmetik durch entsprechende Funktionen realisieren. Dass die Bibliotheken in beiden Fällen zwar dieselben Eingangspunkte haben, darüber hinaus aber sehr verschieden aussehen, sollte offensichtlich sein. Sieht man nun für die mathematischen Funktionen eine besondere Bibliothek vor, so braucht man nur diese in verschiedenen Varianten zu unterhalten, während die „Hauptbibliothek“ nur in einer Variante existiert.

9.3 Fehlerbehandlung

Die Headerdatei `errno.h` enthält zwar nur drei Deklarationen, bietet aber trotzdem interessante Möglichkeiten:

Die Variable `errno` wird von vielen Bibliotheksfunktionen genutzt, um eventuelle Fehler zu markieren. Besitzt `errno` den Wert Null, so bedeutet das, dass kein Fehler aufgetreten ist, während positive Werte Fehler anzeigen.

Allerdings: `errno` wird nur einmal automatisch auf Null zurückgesetzt, nämlich beim Start des Programms. Möchte man eine bestimmte Operation auf fehlerfreien Ablauf hin überwachen, so ist es zweckmäßig, `errno` vor der Operation selbst auf Null zurückzusetzen.

Welche Fehler durch welche Nummern gekennzeichnet werden, überlässt der Standard den Implementatoren. Er schreibt jedoch eine Funktion (in `string.h`) vor, mit der man Fehlernummern in Klarschrift umsetzen kann. Außerdem erlauben die beiden anderen Deklarationen in `errno.h` die direkte Behandlung der häufigsten Fehler, nämlich Fehler bei den mathematischen Funktionen:

- **EDOM** ist der Wert, den `errno` erhält, wenn ein Argument außerhalb des zulässigen Wertebereichs liegt. Der Funktionswert der mathematischen Funktion ist in diesem Falle undefiniert.
- **ERANGE** ist der Wert, den `errno` erhält, wenn der Funktionswert außerhalb des zulässigen Wertebereichs liegt. Der Funktionswert der mathematischen Funktion wird dann

auf den Wert `HUGE_VAL` gesetzt, mit dem korrekten Vorzeichen. `HUGE_VAL` ist in `math.h` deklariert.

Durch den Aufruf `sqrt (- 2.0)` würde etwa `errno` auf den Wert `EDOM` und durch den Aufruf `exp (2000.0)` auf den Wert `ERANGE` gesetzt. Beim ersten Aufruf wäre der Funktionswert undefiniert, beim zweiten `HUGE_VAL` (vergleichbar mit $\pm\infty$).

Damit `errno` in allem Modulen eines komplexen Programms zur Verfügung stehen kann, darf die Variable mit nur einem Speicherplatz verbunden sein. Darum ist die Variable in `errno.h` extern deklariert und steht in allen Modulen zur Verfügung, die `errno.h` durch `#include` einbinden.

Sogar nur eine Deklaration enthält die Headerdatei `assert.h`, die zum Test von Programmen dient: Die Funktion

```
void assert(int bedingung);
```

bewirkt nichts, wenn *bedingung* den Wert *wahr* besitzt. Sollte der Wert *falsch* sein, wird das Programm mit einer Fehlermeldung abgebrochen.

Eine zweite Definition wird in der Headerdatei zwar verwendet, ist aber nicht in ihr enthalten: Wenn der Name `NDEBUG` *nicht* definiert ist, werden die Tests wie beschrieben ausgeführt; ist der Name dagegen definiert, *unterbleiben* die Tests. Das erlaubt, die Aufrufe von `assert` im Quellcode stehen zu lassen, obwohl sie nicht ausgeführt werden sollen – vielleicht braucht man sie nach Änderungen am Programm ja noch einmal. Man muss nur eine Definition für `NDEBUG` ins Programm einfügen. Namen werden mit der `#define`-Direktive definiert; auf Details geht das nächste Kapitel ein.

9.4 Elementare Typen

Die Typdeklarationen und Konstantendefinitionen, die `stddef.h` enthält, sind weitgehend auch in anderen Headerdateien enthalten. Auf einige dieser Deklarationen werden wir noch zurückkommen.

9.5 Diverse Hilfsroutinen

Wundert man sich bei mancher Headerdatei über die Zusammenstellung der Deklarationen, so enthält insbesondere `stdlib.h` ein ziemlich buntes Gemisch von Deklarationen:

- Es gibt Funktionen, die den Inhalt eines String als Zahl interpretieren – z.B. `atoi`.

```
int atoi(const char *s);
```

Daneben gibt es noch eine Reihe weiterer Umwandlungsfunktionen.

- Es gibt Funktionen zur dynamischen Bereitstellung von Speicher – und natürlich auch zur Freigabe solchen Speichers. Wegen ihrer besonderen Bedeutung ist ihnen ein extra Kapitel gewidmet.
- Zwei Funktionen (`abort` und `exit`) erlauben die Beendigung eines Programms an beliebiger Stelle, also ohne geschachtelte Funktionsaufrufe erst bis ins Hauptprogramm zurückverfolgen zu müssen:

```
void abort(void);
void exit(int status);
```

- Mit `getenv` kann man sich Informationen über die Betriebssystemumgebung beschaffen oder mit `system` die Ausführung von Betriebssystemkommandos verlangen.

- Die Funktion `qsort` sortiert einen Vektor; mit `bsearch` kann man in einem sortierten Vektor nach einer Komponente mit einem bestimmten Wert suchen.
- Es gibt Funktionen, die die Verarbeitung erweiterter Zeichensätze erlauben.

Zwei Gruppen von Funktionen aus `stdlib.h` fehlen in diesem sehr pauschalen Überblick. Sie sollen jetzt ein wenig ausführlicher behandelt werden:

Die Funktion

```
int rand(void);
```

realisiert einen *Pseudo-Zufallszahlen-Generator*: Sie liefert bei jedem Aufruf eine ganze Zahl aus dem Bereich $[0, \text{RAND_MAX}]$, wobei $\text{RAND_MAX} \geq 32767 (= 2^{15} - 1)$ gelten muss.

Von einem „Pseudo“-Zufallszahlen-Generator spricht man, weil die Zahlen, die `rand` liefert, nicht wirklich zufällig sind. Vielmehr gibt es eine bestimmte Formel, wie aus dem Startwert bzw. dem letzten gelieferten Wert die nächste zu liefernde Zahl zu berechnen ist.

Pseudo-Zufallszahlen sind zum Programmtest oft nützlich. Betrachten wir als Beispiel den Test einer Sortierfunktion: Man möchte zwar möglichst gut verteilte Zahlen haben, wofür der Generator sorgt – man kann aber auch nicht bei jedem Test andere Werte brauchen, weil eventuelle Fehler reproduzierbar sein müssen. Um verschiedene Zahlenfolgen erzeugen zu können, kann man den Startwert (standardmäßig 1) mit der Funktion `srand` angeben; ihr Prototyp ist

```
void srand(unsigned int start);
```

Steckt man hier etwa Sekunden und Minuten der aktuellen Uhrzeit hinein, so lassen sich die Zahlenfolgen ohne weiteres nicht mehr nachvollziehen. Zumindest werden die Zahlen so „zufällig“, dass sie für Spiele und ähnliche Programme vollständig ausreichen.

Genau genommen liefert `rand` „gleich verteilte“ Zahlen, d.h. die Zahlen sind gleichmäßig über den erlaubten Wertebereich verteilt. Braucht man für einen Test etwa Zahlen mit einer Gauß-Verteilung, so kann man `rand` *nicht* ohne weiteres verwenden.

Zur zweiten Gruppe gehören vier Funktionen zur ganzzahligen Arithmetik: Die Funktionen `abs` und `labs` haben als Argument einen `int`- bzw. `long`-Ausdruck und liefern als Funktionswert dessen Absolutbetrag; die Funktionen `div` und `ldiv` haben als Argumente zwei `int`- bzw. `long`-Ausdrücke und liefern als Funktionswert sowohl den ganzzahligen Quotienten als auch den Rest – wie man erreichen kann, dass Funktionswerte aus mehreren Einzelwerten bestehen, werden wir noch kennenlernen. Das Stichwort ist „Strukturen“.

9.6 Termine und Zeiten

Die verwendete Prozessorzeit des aktuellen Prozesses liefert die Funktion

```
clock_t clock(void);
```

die in `time.h` deklariert ist. Dabei ist `clock_t` ein ganzzahliger Typ, der ebenfalls in `time.h` deklariert ist. Dies geschieht durch eine `typedef`-Deklaration – eine typische Anwendung von `typedef`.

Dieses ist erneut eine typische Vorgehensweise des Standard: Vorgeschrieben wird zwar ein ganzzahliger Typ, nicht jedoch ein konkreter ganzzahliger Typ. Das erlaubt den Implementatoren, zwischen `short`, `int` und `long` zu wählen, abhängig von der Hardware des Rechners, auf dem die Implementation laufen soll. Für den Programmierer wird die konkrete Wahl durch den speziellen Typ weitestgehend transparent, ohne dass sich Restriktionen

ergeben: Bei arithmetischen Operationen sorgt der Compiler für die entsprechenden Umwandlungen! Probleme gibt es allenfalls bei der Ausgabe mit `printf`, weil man nicht weiß, welchen Formatbeschreiber man zu verwenden hat; das lässt sich aber auch lösen, indem man den Wert hierfür zunächst in `long` umwandelt.

Die Maßeinheit von `clock` ist implementations-spezifisch. In Sekunden kann man den Funktionswert von `clock` umrechnen, indem man ihn durch die ebenfalls in `time.h` definierte Konstante `CLOCKS_PER_SEC` dividiert.

Ebenso lässt der Standard offen, zu welchem Zeitpunkt die Zeitmessung gestartet wurde. Es handelt sich dabei um Implementationsabhängiges Verhalten, das entsprechend in der jeweiligen Dokumentation der Implementation beschrieben wird. Mögliche Zeitpunkte für den Start der Zeitmessung sind der Zeitpunkt des Programmstarts oder der erste Aufruf von `clock` im Programm.

Die übrigen Funktionen aus `time.h`, etwa die Bestimmung von Datum und Uhrzeit, erfordern Kenntnisse, die wir derzeit noch nicht haben.